Katowice 2026-04-20

Programowanie obliczeń komputerowych

Laboratorium 1

Sebastian Legierski 301954
Informatyka Przemysłowa, Stopień II, Semestr 2
Wydział Inżynierii Materiałowej i Cyfryzacji Przemysłu, Politechnika Śląska

Spis treści

Wprowadzenie 3
Zadanie moA02zad1 3
Zadanie moA02zad2a 4
Zadanie moA02zad2b 6
Zadanie moA02zad2c 7
Zadanie moA02zad3a 9
Zadanie moA02zad3b 10
Zadanie moA02zad3c 12
Zadanie moA03zad2a 13
Zadanie moA03zad3 15
Zadanie moA03zad4 17
Zadanie moA04zad1a 19
Zadanie moA04zad1b 20
Zadanie moA04zad2a 22
Zadanie moA04zad2b 24
Zadanie moA04zad2c 26
Zadanie moA04zad3a 27
Zadanie moA04zad3b 29
Zadanie moA05zad1a 31
Zadanie moA05zad1b 33
Zadanie moA05zad2 34

Wprowadzenie

Rozwiązania są przygotowane w formie potomków klasy interface'u IZadanie która odpowiedzialna jest za przygotowanie Writer'a wypisującego wyniki do Pliku i/lub Konsoli w zależności od ustawień środowiska, oraz mierzenie czasu pracy kodu poszczególnych zadań. Ułatwia to i organizuje pracę, oraz sprawia, że pisząc rozwiązania do kolejnych zadań można się skupić jedynie na kodzie do rozwiązania, ponieważ do interface'u oddelegowany jest powtarzalny kod. W kodzie poniżej umieszczone zostają jedynie zawartości metody Run(), która implementuje rozwiązanie poszczególnych zadań, oraz ewentualne metody pomocnicze.

W celu zachowania zwięzłości sprawozdania, niektóre serie wyników zostały skrócone o wartości środkowe. Oryginalne pliki są dostępne w archiwum dołączonym do sprawozdania oraz są możliwe do odtworzenia używając kodu źródłowego który również zamieszczony jest w archiwum.

Zadanie moA02zad1

Treść zadania

W poleceniu mamy przygotować program, który oblicza wartości wielomianu piątego stopnia dla kolejnych wartości argumentu x z podanego przedziału i wypisuje wynik w formie tabeli.

Analiza polecenia

Najprościej byłoby zapisać każdy składnik wielomianu osobno, ale wtedy program wykonywałby niepotrzebnie dużo potęgowań. W rozwiązaniu zastosowałem schemat Hornera, czyli zapis od najwyższej potęgi, gdzie kolejne współczynniki są dodawane po przemnożeniu wyniku częściowego przez x. Dzięki temu obliczenia są krótsze i bardziej naturalne dla programu obliczeniowego.

Kod

```
public void Run(TextWriter writer)
{
    writer.WriteLine("| {0,3} | {1,7} |", "x", "y");
    writer.WriteLine("+-{0,3}-+-{1,7}-+", "---", "-------");

    for (double x = _xStart; x <= _xEnd + (_xStep / 100.0); x += _xStep)
    {
        writer.WriteLine("| {0,3:f1} | {1,7:f3} |", x, F1(x));
    }
}

private static double F1(double x)
{
    return ((((1.2 * x + 3.14) * x + 2.58) * x + 0.27) * x + 6.08) * x + 4.5;
}
```

Wynik

```
==================================
ZADANIE moA02zad1
----------------------------------
|   x |       y |
+-----+---------+
| 0.0 |   4.500 |
| 0.1 |   5.114 |
| 0.2 |   5.753 |
| 0.3 |   6.446 |
| 0.4 |   7.233 |
| 0.5 |   8.164 |
| 0.6 |   9.303 |
| 0.7 |  10.729 |
| 0.8 |  12.537 |
| 0.9 |  14.840 |
| 1.0 |  17.770 |

----------------------------------
rozwiazanie zajelo: 12.742 ms
==================================
```

Wnioski

Funkcja poprawnie wyznacza wartości y dla x od 0.0 do 1.0 z krokiem 0.1. Wyniki rosną od 4.500 do 17.770, co jest zgodne z postacią wielomianu w tym przedziale. Czas wykonania: 12.742 ms.

Zadanie moA02zad2a

Treść zadania

W poleceniu mamy obliczyć wartość pierwiastka kwadratowego z liczby b metodą iteracyjną i sprawdzić zbieżność wzoru dla przyjętego x0 oraz liczby kroków n.

Analiza polecenia

Zastosowałem klasyczną iterację Herona, czyli x = 0.5 * (x + b / x). Jest to dobry wybór, ponieważ korzysta tylko z prostych działań arytmetycznych, a jednocześnie bardzo szybko zbliża się do wartości pierwiastka. Program wypisuje kolejne przybliżenia w tabeli, co pozwala łatwo obserwować zbieżność.

Kod

```
    writer.WriteLine("b = {0,2:f1}  x0 = {1,2:f1}  n = {2}\n", _b, _x0, _n);
    writer.WriteLine("| {0,3} | {1,12} |", "i", "x");
    writer.WriteLine("+-{0,3}-+-{1,12}-+", "---", "------------");

    double x = _x0;
    writer.WriteLine("| {0,3} | {1,12:f10} |", 0, x);

    for (int i = 1; i < _n; i++)
    {
        x = 0.5 * (x + _b / x);
        writer.WriteLine("| {0,3} | {1,12:f10} |", i, x);
    }
```

Wynik

```
====================================
ZADANIE moA02zad2a
------------------------------------
b = 2.0  x0 = 1.5  n = 10

|   i |            x |
+-----+--------------+
|   0 | 1.5000000000 |
|   1 | 1.4166666667 |
|   2 | 1.4142156863 |
|   3 | 1.4142135624 |
|   4 | 1.4142135624 |
|   5 | 1.4142135624 |
|   6 | 1.4142135624 |
|   7 | 1.4142135624 |
|   8 | 1.4142135624 |
|   9 | 1.4142135624 |

------------------------------------
rozwiazanie zajelo: 1.304 ms
====================================
```

Wnioski

Dla b = 2 i x0 = 1.5 już po kilku iteracjach wynik stabilizuje się na wartości 1.4142135624, czyli praktycznie na sqrt(2). Metoda jest zbieżna i bardzo efektywna. Czas wykonania: 1.304 ms.

Zadanie moA02zad2b

Treść zadania

W poleceniu mamy sprawdzić drugi wzór iteracyjny dla tego samego równania pierwiastkowego i porównać zachowanie kolejnych przybliżeń.

Analiza polecenia

W tym wariancie iteracja ma postać x = b / x. Algebraicznie wzór nadal wynika z równania x^2 = b, ale nie oznacza to automatycznie dobrej zbieżności numerycznej. Program pokazuje kolejne wartości x, żeby można było sprawdzić, czy ciąg przybliżeń zbliża się do rozwiązania, czy tylko oscyluje.

Kod

```
        writer.WriteLine("b = {0,2:f1}  x0 = {1,2:f1}  n = {2}\n", _b, _x0, _n);
        writer.WriteLine("| {0,3} | {1,12} |", "i", "x");
        writer.WriteLine("+-{0,3}-+-{1,12}-+", "---", "------------");

        double x = _x0;
        writer.WriteLine("| {0,3} | {1,12:f10} |", 0, x);

        for (int i = 1; i < _n; i++)
        {
            x = _b / x;
            writer.WriteLine("| {0,3} | {1,12:f10} |", i, x);
        }
```

Wynik

```
==================================
ZADANIE moA02zad2b
----------------------------------
b = 2.0  x0 = 1.5  n = 10

|   i |           x |
+-----+-------------+
|   0 | 1.5000000000 |
|   1 | 1.3333333333 |
|   2 | 1.5000000000 |
|   3 | 1.3333333333 |
|   4 | 1.5000000000 |
|   5 | 1.3333333333 |
|   6 | 1.5000000000 |
|   7 | 1.3333333333 |
|   8 | 1.5000000000 |
|   9 | 1.3333333333 |

----------------------------------
rozwiazanie zajelo: 0.881 ms
==================================
```

Wnioski

Wynik pokazuje, że metoda nie zbiega do sqrt(2), tylko przeskakuje między wartościami 1.5 oraz 1.3333333333. Samo przekształcenie równania jest poprawne algebraicznie, ale jako iteracja numeryczna jest słabe. Czas wykonania: 0.881 ms.

Zadanie moA02zad2c

Treść zadania

W poleceniu mamy porównać wartości pierwiastka otrzymane z iteracji Herona z dokładnymi wartościami funkcji Math.Sqrt dla b z przedziału od 1.0 do 2.0.

Analiza polecenia

Program wykonuje tę samą iteracje co w zadaniu 2a, ale dla wielu wartości b. Dzięki temu można zobaczyć, jak jedno przybliżenie początkowe x0 = 1.5 i stała liczba iteracji n = 2 działają dla całego zakresu danych. Porównanie z Math.Sqrt pełni rolę kontroli dokładności.

Kod

```
    writer.WriteLine("x0 = {0,2:f1}  n = {1}\n", _x0, _n);
    writer.WriteLine("| {0,4} | {1,12} | {2,12} |", "b", "x", "sqrt(b)");
    writer.WriteLine("+-{0,4}-+-{1,12}-+-{2,12}-+", "----", "--…--", "--…--");

    for (double b = _bStart; b <= _bEnd + (_bStep / 100.0); b += _bStep)
    {
        double x = _x0;
        for (int i = 0; i < _n; i++)
        {
            x = 0.5 * (x + b / x);
        }
        writer.WriteLine("| {0,4:f1} | {1,12:f10} | {2,12:f10} |",
                         b, x, Math.Sqrt(b));
    }
```

Wynik

```
==================================
ZADANIE moA02zad2c
----------------------------------
x0 = 1.5  n = 2

|    b |            x |      sqrt(b) |
+------+--------------+--------------+
|  1.0 | 1.0032051282 | 1.0000000000 |
|  1.1 | 1.0508706468 | 1.0488088482 |
|  1.2 | 1.0967391304 | 1.0954451150 |
|  1.3 | 1.1409624413 | 1.1401754251 |
|  1.4 | 1.1836757991 | 1.1832159566 |
|  1.5 | 1.2250000000 | 1.2247448714 |
|  1.6 | 1.2650432900 | 1.2649110641 |
|  1.7 | 1.3039029536 | 1.3038404810 |
|  1.8 | 1.3416666667 | 1.3416407865 |
|  1.9 | 1.3784136546 | 1.3784048752 |
|  2.0 | 1.4142156863 | 1.4142135624 |

----------------------------------
rozwiazanie zajelo: 0.534 ms
==================================
```

Wnioski

Nawet po dwóch iteracjach wyniki są bardzo bliskie wartościom dokładnym. Dla b = 2 program otrzymuje 1.4142156863, a Math.Sqrt daje 1.4142135624, więc błąd jest mały. Czas wykonania: 0.534 ms.

Zadanie moA02zad3a

Treść zadania

W poleceniu mamy znaleźć nietrywialne rozwiązanie równania x = 2 sin(x) metodą graficzną, czyli przez szukanie punktu przecięcia funkcji f(x) = x oraz g(x) = 2 sin(x).

Analiza polecenia

Rozwiązanie polega na przejściu po przedziale od wartości początkowej do π z małym krokiem dx. Dla każdego punktu program liczy wartości obu funkcji i zatrzymuje się, kiedy ich różnica jest mniejsza lub równa dokładności kroku. Jest to metoda prosta i czytelna, ale jej dokładność zależy bezpośrednio od wielkości dx.

Kod

```
        writer.WriteLine("dx = {0,8:f4}\n", _dx);

        int steps = 0;
        double xFound = _x0;
        double fx = _x0;
        double gx = 2 * Math.Sin(_x0);

        for (double x = _x0; x <= Math.PI; x += _dx)
        {
            steps++;
            fx = x;
            gx = 2 * Math.Sin(x);

            if (Math.Abs(fx - gx) <= _dx)
            {
                xFound = x;
                break;
            }
        }

        writer.WriteLine("{0}) x = {1:f10}", steps, xFound);
        writer.WriteLine("       f(x) = {0:f10}", fx);
        writer.WriteLine("       g(x) = {0:f10}", gx);
```

Wynik

```
==================================
ZADANIE moA02zad3a
----------------------------------
dx =   0.0001

8956) x = 1.8955000000
       f(x) = 1.8955000000
       g(x) = 1.8954906091

----------------------------------
rozwiazanie zajelo: 0.447 ms
==================================
```

Wnioski

Dla dx = 0.0001 znaleziono punkt x = 1.8955000000, gdzie f(x) i g(x) są bardzo blisko siebie. Wynik jest zgodny z oczekiwaniem, ale metoda wymaga wielu kroków przy większej dokładności. Czas wykonania: 0.447 ms.

Zadanie moA02zad3b

Treść zadania

W poleceniu mamy rozwiązać równanie x = 2 sin(x) metodą iteracyjną, używając wzoru x = 2 sin(x), i sprawdzić zachowanie ciągu przybliżeń.

Analiza polecenia

Program startuje od x0 = 1.0 i wykonuje 30 iteracji. Taki zapis jest bezpośrednim przekształceniem badanego równania, więc każda kolejna wartość powinna przybliżać miejsce stałe funkcji 2 sin(x). Na końcu program dodatkowo sprawdza resztę równania, czyli |(2 sin(x) - x) / x|.

Kod

```
    writer.WriteLine("x0 = {0,3:f1}  n = {1}\n", _x0, _n);
    writer.WriteLine("| {0,3} | {1,12} |", "i", "x");
    writer.WriteLine("+-{0,3}-+-{1,12}-+", "---", "------------");

    double x = _x0;
    writer.WriteLine("| {0,3} | {1,12:f10} |", 0, x);

    for (int i = 1; i < _n; i++)
    {
        x = 2 * Math.Sin(x);
        writer.WriteLine("| {0,3} | {1,12:f10} |", i, x);
    }

    writer.WriteLine();
    writer.WriteLine("x = x({0}) = {1:f10}", _n, x);
    writer.WriteLine("|(2*sin(x) - x)/x| = {0:e2}",
                     Math.Abs((2 * Math.Sin(x) - x) / x));
```

Wynik

```
====================================
ZADANIE moA02zad3b
------------------------------------
x0 = 1.0  n = 30

|   i |            x |
+-----+--------------+
|   0 | 1.0000000000 |
|   1 | 1.6829419696 |
|   2 | 1.9874365303 |
...
|  28 | 1.8954950833 |
|  29 | 1.8954937462 |

x = x(30) = 1.8954937462
|(2*sin(x) - x)/x| = 4.50e-007

------------------------------------
rozwiazanie zajelo: 1.636 ms
====================================
```

Wnioski

Iteracja zbiega do wartości około 1.8954937462, a błąd względny równania wynosi 4.50e-007. Metoda działa poprawnie, chociaż widać, że zbieżność jest stopniowa i wymaga większej liczby iteracji. Czas wykonania: 1.636 ms.

Zadanie moA02zad3c

Treść zadania

W poleceniu mamy rozwiązać równanie 2 sin(x) - x = 0 metodą numeryczną opartą o wzór Newtona.

Analiza polecenia

Zastosowałem metodę Newtona, gdzie f(x) = 2 sin(x) - x, a pochodna wynosi f'(x) = 2 cos(x) - 1. W każdym kroku od aktualnej wartości odejmowany jest iloraz f(x) / f'(x). Taka metoda zwykle daje szybką zbieżność, o ile punkt startowy jest dobrany rozsądnie.

Kod

```
        writer.WriteLine("x0 = {0,3:f2}  n = {1}\n", _x0, _n);
        writer.WriteLine("| {0,3} | {1,14} |", "i", "x");
        writer.WriteLine("+-{0,3}-+-{1,14}-+", "---", "--------------");

        double x = _x0;
        writer.WriteLine("| {0,3} | {1,14:f10} |", 0, x);

        for (int i = 1; i < _n; i++)
        {
            double f = 2 * Math.Sin(x) - x;
            double fp = 2 * Math.Cos(x) - 1;
            x -= f / fp;
            writer.WriteLine("| {0,3} | {1,14:f10} |", i, x);
        }

        writer.WriteLine();
        writer.WriteLine("x = x({0}) = {1:f10}", _n, x);
        writer.WriteLine("|(2*sin(x) - x)/x| = {0:e2}",
                         Math.Abs((2 * Math.Sin(x) - x) / x));
```

Wynik

```
==================================
ZADANIE moA02zad3c
----------------------------------
x0 = 1.00  n = 15

|   i |             x |
+-----+---------------+
|   0 |   1.0000000000 |
|   1 |  -7.4727406398 |
|   2 |  14.4785209829 |
|   3 |   6.9351154080 |
|   4 |  16.6356841214 |
|   5 |   8.3439375493 |
|   6 |   4.9546327243 |
|   7 |  -8.3013179977 |
|   8 |  -4.8173199339 |
|   9 |   3.7925744879 |
|  10 |   1.8610609427 |
|  11 |   1.8962142114 |
|  12 |   1.8954945666 |
|  13 |   1.8954942670 |
|  14 |   1.8954942670 |

x = x(15) = 1.8954942670
|(2*sin(x) - x)/x| = 0.00e+000

----------------------------------
rozwiazanie zajelo: 1.591 ms
==================================
```

Wnioski

Otrzymano x = 1.8954942670, a sprawdzenie równania daje błąd zapisany jako 0.00e+000. Wynik jest dokładniejszy niż w metodzie prostej iteracji, co potwierdza skuteczność metody Newtona. Czas wykonania: 1.591 ms.

Zadanie moA03zad2a

Treść zadania

W poleceniu mamy obliczyć błąd bezwzględny objętości kuli, gdy liczba π i średnica d są dane jako wartości przybliżone z określonymi błędami.

Analiza polecenia

Zastosowałem metodę przedziałów. Program przeszukuje siatkę wartości π oraz d w granicach wyznaczonych przez ich błędy i dla każdej pary oblicza objętość V = π * d^3 / 6. Następnie wybierane są najmniejsza i największa uzyskana wartość, co pozwala oszacować rozrzut wyniku.

Kod

```
    public void Run(TextWriter writer)
    {
        writer.WriteLine("V = pi * d^3 / 6");
        writer.WriteLine("pi = {0:f5}, DeltaPi = {1:f5}, dPi = {2:f5}", _pi, _deltaPi, _dPi);
        writer.WriteLine("D  = {0:f5}, DeltaD  = {1:f5}, dD  = {2:f5}", _d, _deltaD, _dD);
        writer.WriteLine();

        double minV = double.MaxValue;
        double maxV = double.MinValue;
        double minPi = _pi;
        double maxPi = _pi;
        double minD = _d;
        double maxD = _d;

        for (double pi = _pi - _deltaPi; pi <= _pi + _deltaPi + (_dPi / 2.0); pi += _dPi)
        {
            for (double d = _d - _deltaD; d <= _d + _deltaD + (_dD / 2.0); d += _dD)
            {
                double v = SphereVolume(pi, d);

                if (v < minV)
                {
                    minV = v;
                    minPi = pi;
                    minD = d;
                }

                if (v > maxV)
                {
                    maxV = v;
                    maxPi = pi;
                    maxD = d;
                }
            }
        }

        writer.WriteLine("Vmin = {0:f10} dla pi = {1:f5}, d = {2:f5}", minV, minPi, minD);
        writer.WriteLine("Vmax = {0:f10} dla pi = {1:f5}, d = {2:f5}", maxV, maxPi, maxD);
        writer.WriteLine("DeltaV = {0:f10}", maxV - minV);
        writer.WriteLine();
    }

    private static double SphereVolume(double pi, double d)
    {
        return pi * d * d * d / 6.0;
    }
```

Wynik

```
==================================
ZADANIE moA03zad2a
----------------------------------
V = pi * d^3 / 6
pi = 3.14000, DeltaPi = 0.01000, dPi = 0.00001
D  = 1.25000, DeltaD  = 0.02000, dD  = 0.00001

Vmin = 0.9707522850 dla pi = 3.13000, d = 1.23000
Vmax = 1.0754010750 dla pi = 3.15000, d = 1.27000
DeltaV = 0.1046487900

----------------------------------
rozwiazanie zajelo: 11.684 ms
==================================
```

Wnioski

Program wyznaczył Vmin oraz Vmax dla skrajnych wartości π i d, a różnica DeltaV wyniosła 0.1046487900. Wynik pokazuje, jak błędy danych wejściowych przenoszą się na wynik obliczania objętości. Czas wykonania: 11.684 ms.

Zadanie moA03zad3

Treść zadania

W poleceniu mamy porównać obliczanie dwóch algebraicznie równoważnych funkcji i sprawdzić, jak zaokrąglenia pośrednich wyników wpływają na końcową dokładność.

Analiza polecenia

W programie liczone są dwie postacie wyrażenia: jedna zawiera odejmowanie bliskich sobie wartości pierwiastków, a druga ma postać przekształconą z dzieleniem przez sumę. Pierwsza metoda jest bardziej narażona na utratę cyfr znaczących, bo przy małych x różnica dwóch podobnych liczb robi się bardzo mała. Druga postać unika tego problemu i dlatego powinna być stabilniejsza numerycznie.

Kod

```
 public void Run(TextWriter writer)
{
    writer.WriteLine("a = {0:f1}  n = {1}\n", _a, _n);
    writer.WriteLine("| {0,8} | {1,13} | {2,13} | {3,6} | {4,6} | {5,6} |", "x", "f(x)", "g(x)", "df", "dg",
                     "dfg");
    writer.WriteLine("+-{0,8}-+-{1,13}-+-{2,13}-+-{3,6}-+-{4,6}-+-{5,6}-+", "--------", "-------------",
                     "-------------", "------", "------", "------");

    for (int i = 0; i < _n; i++)
    {
        double x = GenerateX(_a, i);
        double s1 = Math.Sqrt(1 + x * x);
        double s2 = Math.Sqrt(1 - x * x);

        double s1Rounded = Math.Round(s1, 11);
        double s2Rounded = Math.Round(s2, 7);

        double f = CalculateF(s1Rounded, s2Rounded);
        double g = CalculateG(x, s1Rounded, s2Rounded);

        double deltaS1 = CalculateRelativeRoundingError(s1, 11);
        double deltaS2 = CalculateRelativeRoundingError(s2, 7);

        double deltaF = CalculateDeltaF(s1, s2, f, deltaS1, deltaS2);
        double deltaG = CalculateDeltaG(s1, s2, deltaS1, deltaS2);
        double deltaFG = Math.Abs((g - f) / g);

        writer.WriteLine(
            "| {0,8:f6} | {1,13:e6} | {2,13:e6} | {3,6:e0} | {4,6:e0} | {5,6:e0} |",
            x,
            f,
            g,
            deltaF,
            deltaG,
            deltaFG);
    }

    writer.WriteLine();
}

private static double GenerateX(double a, int step)
{
    return a / Math.Pow(2, step + 2);
}

private static double CalculateF(double s1, double s2)
{
    return s1 - s2;
}

private static double CalculateG(double x, double s1, double s2)
{
    return (2 * x * x) / (s1 + s2);
}

private static double CalculateRelativeRoundingError(double value, int digitsAfterDecimal)
{
    double absoluteErrorBound = 0.5 * Math.Pow(10, -digitsAfterDecimal);
    return absoluteErrorBound / Math.Abs(value);
}

private static double CalculateDeltaF(double s1, double s2, double f, double deltaS1, double deltaS2)
{
    return Math.Abs((s1 / Math.Abs(f)) * deltaS1 + (s2 / Math.Abs(f)) * deltaS2);
}

private static double CalculateDeltaG(double s1, double s2, double deltaS1, double deltaS2)
{
    double denominator = s1 + s2;
    return Math.Abs((s1 / denominator) * deltaS1 + (s2 / denominator) * deltaS2);
}
```

Wynik

```
==================================
ZADANIE moA03zad3
----------------------------------
a = 2.0  n = 15

|        x |          f(x) |          g(x) |     df |     dg |    dfg |
+----------+---------------+---------------+--------+--------+--------+
| 0.500000 | 2.520086e-001 | 2.520086e-001 | 2e-007 | 3e-008 | 1e-008 |
| 0.250000 | 6.253061e-002 | 6.253057e-002 | 8e-007 | 3e-008 | 6e-007 |
| 0.125000 | 1.562552e-002 | 1.562548e-002 | 3e-006 | 3e-008 | 3e-006 |
| 0.062500 | 3.906221e-003 | 3.906257e-003 | 1e-005 | 3e-008 | 9e-006 |
| 0.031250 | 9.765621e-004 | 9.765626e-004 | 5e-005 | 3e-008 | 5e-007 |
| 0.015625 | 2.441629e-004 | 2.441406e-004 | 2e-004 | 3e-008 | 9e-005 |
| 0.007812 | 6.101711e-005 | 6.103516e-005 | 8e-004 | 3e-008 | 3e-004 |
| 0.003906 | 1.522937e-005 | 1.525879e-005 | 3e-003 | 3e-008 | 2e-003 |
| 0.001953 | 3.807350e-006 | 3.814697e-006 | 1e-002 | 3e-008 | 2e-003 |
| 0.000977 | 9.768400e-007 | 9.536743e-007 | 5e-002 | 3e-008 | 2e-002 |
| 0.000488 | 2.192100e-007 | 2.384186e-007 | 2e-001 | 3e-008 | 8e-002 |
| 0.000244 | 2.980000e-008 | 5.960464e-008 | 2e+000 | 3e-008 | 5e-001 |
| 0.000122 | 7.450000e-009 | 1.490116e-008 | 7e+000 | 3e-008 | 5e-001 |
| 0.000061 | 1.860000e-009 | 3.725290e-009 | 3e+001 | 3e-008 | 5e-001 |
| 0.000031 | 4.700000e-010 | 9.313226e-010 | 1e+002 | 3e-008 | 5e-001 |

----------------------------------
rozwiazanie zajelo: 1.562 ms
==================================
```

Wnioski

Wyniki pokazują, że dla małych wartości x funkcja f(x) zaczyna mocno odbiegać od g(x), a względna różnica dochodzi do 5e-001. To dobrze pokazuje problem znoszenia się bliskich składników i utraty dokładności. Czas wykonania: 1.562 ms.

Zadanie moA03zad4

Treść zadania

W poleceniu mamy iteracyjnie obliczać potęgi (1/3)^n podanym schematem rekurencyjnym i porównać je z wartościami dokładnymi z Math.Pow.

Analiza polecenia

Program korzysta z wartości początkowych x0 = 1 oraz x1 = 1/3, a potem liczy kolejne elementy wzorem rekurencyjnym. Taka metoda jest ciekawa, ponieważ teoretycznie opisuje właściwy ciąg, ale numerycznie może wzmacniać małe błędy zaokrągleń. Porównanie z Math.Pow pozwala zobaczyć, od którego momentu metoda przestaje być wiarygodna.

Kod

```
    writer.WriteLine("N = {0}\n", _nMax);
    writer.WriteLine("| {0,3} | {1,14} | {2,14} |", "n", "x", "pow");
    writer.WriteLine("+-{0,3}-+-{1,14}-+-{2,14}-+", "---",
                     "--------------", "--------------");

    double previous = _x0;
    double current = _x1;

    for (int n = 2; n <= _nMax; n++)
    {
        double next = (13.0 / 3.0) * current - (4.0 / 3.0) * previous;
        double exact = Math.Pow(1.0 / 3.0, n);

        writer.WriteLine("| {0,3} | {1,14:e6} | {2,14:e6} |", n, next, exact);

        previous = current;
        current = next;
    }
```

Wynik

```
====================================
ZADANIE moA03zad4
------------------------------------
N = 20

|   n |              x |            pow |
+-----+----------------+----------------+
|   2 |  1.111111e-001 |  1.111111e-001 |
|   3 |  3.703704e-002 |  3.703704e-002 |
|   4 |  1.234568e-002 |  1.234568e-002 |
…
|  19 | -3.402108e-006 |  8.603916e-010 |
|  20 | -1.361159e-005 |  2.867972e-010 |

------------------------------------
rozwiazanie zajelo: 0.672 ms
====================================
```

Wnioski

Do pewnego momentu wyniki są zgodne, ale dla większych n błąd szybko narasta. Dla n = 20 iteracja daje -1.361159e-005, podczas gdy Math.Pow daje 2.867972e-010, więc metoda jest numerycznie niestabilna. Czas wykonania: 0.672 ms.

Zadanie moA04zad1a

Treść zadania

W poleceniu mamy obliczyć przybliżenie liczby e z ciągu (1 + 1/n)^n i sprawdzić jego zbieżność dla dużej wartości N.

Analiza polecenia

Program liczy kolejne wartości ciągu i wypisuje pierwsze kroki oraz co tysięczny wynik, żeby tabela nie była niepotrzebnie długa. Metoda jest bardzo prosta, ale z teorii wiadomo, że zbiega do e powoli. Porównanie końcowego wyniku z Math.E pokazuje praktyczną dokładność tej metody.

Kod

```
public void Run(TextWriter writer)
{
    writer.WriteLine("N = {0}\n", _nMax);
    writer.WriteLine("| {0,6} | {1,14} |", "n", "x");
    writer.WriteLine("+-{0,6}-+-{1,14}-+", "------", "--------------");

    double x = 0.0;
    for (int n = 1; n <= _nMax; n++)
    {
        x = Math.Pow(1.0 + (1.0 / n), n);
        if (ShouldPrint(n))
        {
            writer.WriteLine("| {0,6} | {1,14:f10} |", n, x);
        }
    }

    writer.WriteLine();
    writer.WriteLine("x({0}) = {1:g15}", _nMax, x);
    writer.WriteLine("      e = {0:g15}", Math.E);
    writer.WriteLine();
}

private bool ShouldPrint(int n)
{
    return n <= 10 || n % _summaryStep == 0 || n == _nMax;
}
```

Wynik

```
==================================
ZADANIE moA04zad1a
----------------------------------
N = 20000

|      n |              x |
+--------+----------------+
|      1 |   2.0000000000 |
|      2 |   2.2500000000 |
|      3 |   2.3703703704 |
...
|  18000 |   2.7182063245 |
|  19000 |   2.7182102982 |
|  20000 |   2.7182138745 |

x(20000) = 2.71821387453362
      e = 2.71828182845905

----------------------------------
rozwiazanie zajelo: 2.725 ms
==================================
```

Wnioski

Dla N = 20000 otrzymano 2.71821387453362, a wartość Math.E wynosi 2.71828182845905. Mimo dużej liczby kroków różnica jest jeszcze widoczna, więc metoda ciągowa jest mało zbieżna. Czas wykonania: 2.725 ms.

Zadanie moA04zad1b

Treść zadania

W poleceniu mamy obliczyć przybliżenie liczby e za pomocą szeregu 1 + 1/1! + 1/2! + ... i sprawdzić jego zbieżność.

Analiza polecenia

W programie kolejne wyrazy szeregu są liczone iteracyjnie przez dzielenie poprzedniego wyrazu przez n. Dzięki temu nie trzeba za każdym razem liczyć silni od początku. Suma szeregu powinna szybko zbliżać się do e, bo kolejne wyrazy bardzo szybko maleją.

Kod

```
    writer.WriteLine("N = {0}\n", _nMax);
    writer.WriteLine("| {0,3} | {1,14} |", "n", "x");
    writer.WriteLine("+-{0,3}-+-{1,14}-+", "---", "--------------");

    double sum = 1.0;
    double term = 1.0;
    writer.WriteLine("| {0,3} | {1,14:f10} |", 0, sum);

    for (int n = 1; n <= _nMax; n++)
    {
        term /= n;
        sum += term;
        writer.WriteLine("| {0,3} | {1,14:f10} |", n, sum);
    }

    writer.WriteLine();
    writer.WriteLine("x({0}) = {1:g15}", _nMax, sum);
    writer.WriteLine("    e = {0:g15}", Math.E);
```

Wynik

```
==================================
ZADANIE moA04zad1b
----------------------------------
N = 17

|   n |              x |
+-----+----------------+
|   0 |   1.0000000000 |
|   1 |   2.0000000000 |
|   2 |   2.5000000000 |
...
|  16 |   2.7182818285 |
|  17 |   2.7182818285 |

x(17) = 2.71828182845905
    e = 2.71828182845905

----------------------------------
rozwiazanie zajelo: 1.255 ms
==================================
```

Wnioski

Dla N = 17 wynik jest równy 2.71828182845905, czyli zgadza się z Math.E w wypisanej dokładności. Szereg jest więc zdecydowanie lepszą metodą niż ciąg z zadania 1a. Czas wykonania: 1.255 ms.

Zadanie moA04zad2a

Treść zadania

W poleceniu mamy obliczyć wartość funkcji exp(a) metodą ciągu (1 + a/n)^n dla różnych wartości argumentu a.

Analiza polecenia

To jest uogólnienie metody z poprzedniego zadania, tylko zamiast stałej e liczymy e^a. Program sprawdza osobno przypadki a = 0.1 oraz a = 10.0, ponieważ wielkość argumentu ma duży wpływ na tempo zbieżności. Dla większego a metoda powinna potrzebować znacznie więcej kroków.

Kod

```
public void Run(TextWriter writer)
{
    foreach (var expCase in _cases)
    {
        writer.WriteLine("a = {0:f1}   N = {1}\n", expCase.A, expCase.N);
        writer.WriteLine("| {0,6} | {1,14} |", "n", "x");
        writer.WriteLine("+-{0,6}-+-{1,14}-+", "------", "--------------");

        double x = 0.0;
        for (int n = 1; n <= expCase.N; n++)
        {
            x = Math.Pow(1.0 + (expCase.A / n), n);
            if (ShouldPrint(n, expCase.N, expCase.SummaryStep))
            {
                writer.WriteLine("| {0,6} | {1,14:e7} |", n, x);
            }
        }

        writer.WriteLine();
        writer.WriteLine("x({0}) = {1:g15}", expCase.N, x);
        writer.WriteLine("     exp = {0:g15}", Math.Exp(expCase.A));
        writer.WriteLine();
    }
}

private static bool ShouldPrint(int n, int nMax, int summaryStep)
{
    return n <= 10 || n % summaryStep == 0 || n == nMax;
}
```

Wynik

```
==================================
ZADANIE moA04zad2a
----------------------------------
a = 0.1   N = 20000

|      n |              x |
+--------+----------------+
|      1 | 1.1000000e+000 |
|      2 | 1.1025000e+000 |
|      3 | 1.1033704e+000 |
|      4 | 1.1038129e+000 |
...
|  19000 | 1.1051706e+000 |
|  20000 | 1.1051706e+000 |

x(20000) = 1.1051706417846
     exp = 1.10517091807565

a = 10.0   N = 20000

|      n |              x |
+--------+----------------+
|      1 | 1.1000000e+001 |
|      2 | 3.6000000e+001 |
|      3 | 8.1370370e+001 |
...
|  19000 | 2.1968598e+004 |
|  20000 | 2.1971487e+004 |

x(20000) = 21971.4867083777
     exp = 22026.4657948067

----------------------------------
rozwiazanie zajelo: 4.101 ms
==================================
```

Wnioski

Dla a = 10 i N = 20000 program otrzymał 21971.4867083777, a Math.Exp daje 22026.4657948067. Widać, że metoda ciągowa jest wolno zbieżna, szczególnie dla większych argumentów. Czas wykonania: 4.101 ms.

Zadanie moA04zad2b

Treść zadania

W poleceniu mamy obliczyć exp(a) za pomocą szeregu potęgowego i porównać z wartością Math.Exp(a), także dla ujemnego argumentu.

Analiza polecenia

Program sumuje kolejne wyrazy szeregu Taylora, licząc następny wyraz przez przemnożenie poprzedniego przez a / n. Dla dodatnich argumentów metoda powinna być bardzo dobra. Dla a = -10 pojawiają się jednak naprzemienne duże dodatnie i ujemne składniki, więc wynik może wolniej stabilizować się w pobliżu wartości dokładnej.

Kod

```
        foreach (var expCase in _cases)
        {
            writer.WriteLine("a = {0:f1}   N = {1}\n", expCase.A, expCase.N);
            writer.WriteLine("| {0,6} | {1,15} |", "n", "x");
            writer.WriteLine("+-{0,6}-+-{1,15}-+", "------", "---------------");

            double sum = 1.0;
            double term = 1.0;
            writer.WriteLine("| {0,6} | {1,15:e7} |", 0, sum);

            for (int n = 1; n <= expCase.N; n++)
            {
                term *= expCase.A / n;
                sum += term;
                writer.WriteLine("| {0,6} | {1,15:e7} |", n, sum);
            }

            writer.WriteLine();
            writer.WriteLine("x({0}) = {1:g15}", expCase.N, sum);
            writer.WriteLine("  exp = {0:g15}", Math.Exp(expCase.A));
            writer.WriteLine();
        }
```

Wynik

```
==================================
ZADANIE moA04zad2b
----------------------------------
a = 0.1   N = 10

|      n |              x |
+--------+----------------+
|      0 |  1.0000000e+000 |
|      1 |  1.1000000e+000 |
...
|      9 |  1.1051709e+000 |
|     10 |  1.1051709e+000 |

x(10) = 1.10517091807565
  exp = 1.10517091807565

a = 10.0   N = 40

|      n |              x |
+--------+----------------+
|      0 |  1.0000000e+000 |
|      1 |  1.1000000e+001 |
...
|     39 |  2.2026466e+004 |
|     40 |  2.2026466e+004 |

x(40) = 22026.4657948028
  exp = 22026.4657948067

a = -10.0   N = 40

|      n |              x |
+--------+----------------+
|      0 |  1.0000000e+000 |
|      1 | -9.0000000e+000 |
...
|     39 |  4.5390086e-005 |
|     40 |  4.5402342e-005 |

x(40) = 4.5402342005348e-05
  exp = 4.53999297624849e-05

----------------------------------
rozwiazanie zajelo: 2.082 ms
==================================
```

Wnioski

Dla a = 0.1 i a = 10 wyniki są bardzo dobre, natomiast dla a = -10 szereg oscyluje i kończy na 4.5402342005348e-05 przy wartości dokładnej 4.53999297624849e-05. Metoda działa, ale dla dużych ujemnych argumentów jest mniej wygodna numerycznie. Czas wykonania: 2.082 ms.

Zadanie moA04zad2c

Treść zadania

W poleceniu mamy poprawić obliczanie exp(a) dla ujemnego argumentu, korzystając z zależności exp(a) = 1 / exp(-a).

Analiza polecenia

Zamiast sumować szereg dla liczby ujemnej, program liczy szereg dla dodatniego argumentu -a, a następnie bierze odwrotność sumy. Dzięki temu unika się naprzemiennego dodawania i odejmowania dużych wyrazów, które było problemem w poprzednim wariancie. To daje stabilniejsze obliczenia dla a = -10.

Kod

```
        foreach (var expCase in _cases)
        {
            writer.WriteLine("a = {0:f1}   N = {1}\n", expCase.A, expCase.N);
            writer.WriteLine("| {0,6} | {1,15} |", "n", "y");
            writer.WriteLine("+-{0,6}-+-{1,15}-+", "------", "---------------");

            double positiveArgument = -expCase.A;
            double sum = 1.0;
            double term = 1.0;
            double y = 1.0 / sum;
            writer.WriteLine("| {0,6} | {1,15:e7} |", 0, y);

            for (int n = 1; n <= expCase.N; n++)
            {
                term *= positiveArgument / n;
                sum += term;
                y = 1.0 / sum;
                writer.WriteLine("| {0,6} | {1,15:e7} |", n, y);
            }

            writer.WriteLine();
            writer.WriteLine("y({0}) = {1:g15}", expCase.N, y);
            writer.WriteLine("  exp = {0:g15}", Math.Exp(expCase.A));
            writer.WriteLine();
        }
```

Wynik

```
==================================
ZADANIE moA04zad2c
----------------------------------
a = -10.0   N = 40

|      n |               y |
+--------+-----------------+
|      0 |  1.0000000e+000 |
|      1 |  9.0909091e-002 |
|      2 |  1.6393443e-002 |
|      3 |  4.3923865e-003 |
...
|     38 |  4.5399930e-005 |
|     39 |  4.5399930e-005 |
|     40 |  4.5399930e-005 |

y(40) = 4.53999297624929e-05
  exp = 4.53999297624849e-05

----------------------------------
rozwiazanie zajelo: 1.038 ms
==================================
```

Wnioski

Dla a = -10 otrzymano 4.53999297624929e-05, a Math.Exp daje 4.53999297624849e-05. Różnica jest praktycznie pomijalna, więc przekształcenie znacząco poprawia stabilność obliczeń. Czas wykonania: 1.038 ms.

Zadanie moA04zad3a

Treść zadania

W poleceniu mamy przybliżyć liczbę pi metodą stochastyczną, losując punkty w kwadracie i zliczając, ile z nich wpada do wpisanego koła.

Analiza polecenia

Zastosowałem metodę Monte Carlo. Program generuje punkty z przedziału [-0.5, 0.5] dla obu współrzędnych i sprawdza warunek x^2 + y^2 <= 0.25. Stosunek liczby punktów w kole do liczby wszystkich losowań pozwala oszacować pi ze wzoru 4 * No / N.

Kod

```
        var random = new Random(_seed);
        int insideCircle = 0;
        int targetIndex = 0;

        for (int i = 1; i <= _sampleSizes[^1]; i++)
        {
            double x = random.NextDouble() - 0.5;
            double y = random.NextDouble() - 0.5;

            if ((x * x) + (y * y) <= 0.25)
            {
                insideCircle++;
            }

            if (i == _sampleSizes[targetIndex])
            {
                double approximation = 4.0 * insideCircle / i;
                writer.WriteLine("w({0}) = {1:g15}", i, approximation);
                targetIndex++;

                if (targetIndex == _sampleSizes.Length)
                {
                    break;
                }
            }
        }

        writer.WriteLine("       pi = {0:g15}", Math.PI);
```

Wynik

```
==================================
ZADANIE moA04zad3a
----------------------------------
w(1000) = 3.188
w(10000) = 3.1296
w(100000) = 3.14236
w(1000000) = 3.14302
w(10000000) = 3.1419072
       pi = 3.14159265358979

----------------------------------
rozwiazanie zajelo: 108.624 ms
==================================
```

Wnioski

Wraz ze wzrostem liczby losowań wynik zbliża się do pi, ale nadal widać losowe odchylenia. Dla 10 000 000 punktów otrzymano 3.1419072 przy dokładnej wartości 3.14159265358979, więc metoda wymaga bardzo wielu prób. Czas wykonania: 108.624 ms.

Zadanie moA04zad3b

Treść zadania

W poleceniu mamy obliczyć liczbę pi z szeregu Bailey-Borwein-Plouffe i porównać wynik z Math.PI.

Analiza polecenia

Szereg BBP ma bardzo szybką zbieżność, dlatego w programie wystarczy policzyć niewiele wyrazów. Każdy wyraz ma czynnik 1 / 16^n oraz kombinację kilku ułamków, więc kolejne poprawki szybko stają się bardzo małe. Jest to dużo bardziej efektywne niż metoda losowa.

Kod

```
public void Run(TextWriter writer)
{
    writer.WriteLine("N = {0}\n", _nMax);
    writer.WriteLine("| {0,6} | {1,19} |", "n", "x");
    writer.WriteLine("+-{0,6}-+-{1,19}-+", "------", "-------------------");

    double sum = 0.0;
    for (int n = 0; n <= _nMax; n++)
    {
        sum += BaileyBorweinPlouffeTerm(n);
        writer.WriteLine("| {0,6} | {1,19:f15} |", n, sum);
    }

    writer.WriteLine();
    writer.WriteLine("x({0}) = {1:g15}", _nMax, sum);
    writer.WriteLine("   pi = {0:g15}", Math.PI);
    writer.WriteLine();
}

private static double BaileyBorweinPlouffeTerm(int n)
{
    double factor = 1.0 / Math.Pow(16.0, n);
    return factor * (
        4.0 / ((8.0 * n) + 1.0) -
        2.0 / ((8.0 * n) + 4.0) -
        1.0 / ((8.0 * n) + 5.0) -
        1.0 / ((8.0 * n) + 6.0));
}
```

Wynik

```
==================================
ZADANIE moA04zad3b
----------------------------------
N = 10

|      n |                  x |
+--------+--------------------+
|      0 |   3.133333333333333 |
|      1 |   3.141422466422466 |
|      2 |   3.141587390346582 |
|      3 |   3.141592457567436 |
|      4 |   3.141592645460336 |
|      5 |   3.141592653228088 |
|      6 |   3.141592653572881 |
|      7 |   3.141592653588973 |
|      8 |   3.141592653589752 |
|      9 |   3.141592653589791 |
|     10 |   3.141592653589793 |

x(10) = 3.14159265358979
   pi = 3.14159265358979

----------------------------------
rozwiazanie zajelo: 1.409 ms
==================================
```

Wnioski

Dla N = 10 wynik wynosi 3.14159265358979, czyli zgadza się z Math.PI w wypisanej dokładności. Szereg BBP jest zdecydowanie szybszy i dokładniejszy niż metoda Monte Carlo z poprzedniego zadania. Czas wykonania: 1.409 ms.

Zadanie moA05zad1a

Treść zadania

W poleceniu mamy znaleźć precyzję maszynową dla typu float przez kolejne dzielenie liczby x przez 2 i sprawdzanie warunku 1 + x == 1.

Analiza polecenia

Program zaczyna od wartości x = 2 i w pętli zmniejsza ją przez dzielenie przez 2. Gdy dodanie aktualnego x do 1 nie zmienia już wyniku, oznacza to, że x jest zbyt małe, aby zostać rozróżnione w arytmetyce float. Za precyzję przyjmowana jest poprzednia wartość, czyli x pomnożone przez dzielnik.

Kod

```
    writer.WriteLine("a = {0}\n", _a);
    writer.WriteLine("d = {0}\n", _d);

    float x = _a;

    while (true)
    {
        writer.Write($"x = {x}");
        if(x + 1 == 1)
        {
            writer.WriteLine(" => 1+x jest rowne 1 !!!");
            writer.WriteLine($"precyzja dla typu float wynosi, eps = { x * _d }");
            break;
        }
        writer.WriteLine($"");
        x /= _d;
    }
```

Wynik

```
==================================
ZADANIE moA05zad1a
----------------------------------
a = 2

d = 2

x = 2
x = 1
x = 0.5
x = 0.25
...
x = 4.7683716E-07
x = 2.3841858E-07
x = 1.1920929E-07
x = 5.9604645E-08 => 1+x jest rowne 1 !!!
precyzja dla typu float wynosi, eps = 1.1920929E-07
----------------------------------
rozwiazanie zajelo: 1.898 ms
==================================
```

Wnioski

Dla typu float program wyznaczył eps = 1.1920929E-07. Wynik odpowiada około 7 cyfrom znaczącym, co jest typowe dla pojedynczej precyzji. Czas wykonania: 1.898 ms.

Zadanie moA05zad1b

Treść zadania

W poleceniu mamy znaleźć precyzję maszynową dla typu double analogicznie jak dla typu float.

Analiza polecenia

Algorytm jest taki sam jak w poprzednim zadaniu, ale zmienne są typu double. Dzięki większej liczbie bitów mantysy pętla może zejść do znacznie mniejszych wartości x, zanim dodanie do 1 przestanie być widoczne. Porównanie z float pokazuje praktyczną różnicę między pojedynczą i podwójną precyzją.

Kod

```
writer.WriteLine("a = {0}\n", _a);
writer.WriteLine("d = {0}\n", _d);

double x = _a;

while (true)
{
    writer.Write($"x = {x}");
    if (x + 1 == 1)
    {
        writer.WriteLine(" => 1+x jest rowne 1 !!!");
        writer.WriteLine($"precyzja dla typu double wynosi, eps = {x * _d}");
        break;
    }
    writer.WriteLine($"");
    x /= _d;
}
```

Wynik

```
==================================
ZADANIE moA05zad1b
----------------------------------
a = 2

d = 2

x = 2
x = 1
x = 0.5
x = 0.25
...
x = 4.440892098500626E-16
x = 2.220446049250313E-16
x = 1.1102230246251565E-16 => 1+x jest rowne 1 !!!
precyzja dla typu double wynosi, eps = 2.220446049250313E-16
----------------------------------
rozwiazanie zajelo: 4.365 ms
==================================
```

Wnioski

Dla typu double program wyznaczył eps = 2.220446049250313E-16. Jest to dużo mniejsza wartość niż dla float, więc double pozwala zachować około 15-16 cyfr znaczących. Czas wykonania: 4.365 ms.

Zadanie moA05zad2

Treść zadania

W poleceniu mamy sprawdzić, czy w arytmetyce typu float dwa algebraicznie równoważne sposoby dodawania trzech liczb dają dokładnie ten sam wynik.

Analiza polecenia

Program liczy w1 = a + (b + c) oraz w2 = (a + b) + c. W matematyce te wyrażenia są równe, ale w arytmetyce zmiennopozycyjnej kolejność działań może zmienić wynik przez zaokrąglenia. Dlatego najpierw sprawdzana jest równość dokładna, a potem równość przybliżona z użyciem eps dla typu float.

Kod

```
public void Run(TextWriter writer)
{
    float w1 = _a + (_b + _c);
    float w2 = (_a + _b) + _c;

    WriteValue(writer, "a", _a, "0.00000000000");
    WriteValue(writer, "b", _b, "0.00000");
    WriteValue(writer, "c", _c, "0.00000");
    WriteValue(writer, "w1", w1, "0.00000");
    WriteValue(writer, "w2", w2, "0.00000");
    writer.WriteLine();

    if (w1 == w2)
    {
        writer.WriteLine("w1 jest dokladnie rowne w2");
        return;
    }

    writer.WriteLine("w1 nie jest dokladnie rowne w2");

    if (AreApproximatelyEqual(w1, w2))
    {
        writer.WriteLine("w1 jest w przyblizeniu rowne w2");
    }
    else
    {
        writer.WriteLine("w1 jest istotnie rozne od w2");
    }
}

private static bool AreApproximatelyEqual(float x, float y)
{
    if (x == y)
    {
        return true;
    }

    if (Sign(x) != Sign(y))
    {
        return false;
    }

    float scale = MathF.Max(MathF.Abs(x), MathF.Abs(y));
    if (scale == 0f)
    {
        return true;
    }

    return MathF.Abs(x - y) <= Eps * scale;
}

private static int Sign(float x)
{
    if (x > 0f)
    {
        return 1;
    }

    if (x < 0f)
    {
        return -1;
    }

    return 0;
}

private static void WriteValue(TextWriter writer, string label, float value, string format)
{
    string text = value.ToString(format, PolishCulture);
    writer.WriteLine($"{label,-2} = {text,15}");
}
```

Wynik

```
==================================
ZADANIE moA05zad2
----------------------------------
a  =   0,00010234880
b  =       236,36100
c  =      -509,27890
w1 =      -272,91780
w2 =      -272,91780

w1 nie jest dokladnie rowne w2
w1 jest w przyblizeniu rowne w2
----------------------------------
rozwiazanie zajelo: 2.445 ms
==================================
```

Wnioski

Wyniki w1 i w2 nie są dokładnie równe, ale mieszczą się w granicach porównania przybliżonego. To pokazuje, że prawo łączności dodawania nie zawsze działa dosłownie w arytmetyce float, chociaż różnica może być numerycznie nieistotna. Czas wykonania: 2.445 ms.